HG-PIPE: Vision Transformer Acceleration with Hybrid-Grained Pipeline

Qingyu Guo[1], Jiayong Wan[3], Songqiang Xu[3], Meng Li[2,1,4*], Yuan Wang[1,4*]
[1]School of Integrated Circuits, Peking University, China [2]Institute for Artificial Intelligence, Peking University, China
[3]School of Software and Microelectronics, Peking University, China
[4]Beijing Advanced Innovation Center for Integrated Circuits, China
*meng.li@pku.edu.cn, *wangyuan@pku.edu.cn

ABSTRACT

Vision Transformer (ViT) acceleration with field programmable gate array (FPGA) is promising but challenging. Existing FPGA-based ViT accelerators mainly rely on temporal architectures, which process different operators by reusing the same hardware blocks and suffer from extensive memory access overhead. Pipelined architectures, either coarse-grained or fine-grained, unroll the ViT computation spatially for memory access efficiency. However, they usually suffer from significant hardware resource constraints and pipeline bubbles induced by the global computation dependency of ViT. In this paper, we introduce HG-PIPE, a pipelined FPGA accelerator for high-throughput and low-latency ViT processing. HG-PIPE features a hybrid-grained pipeline architecture to reduce on-chip buffer cost and couples the computation dataflow and parallelism design to eliminate the pipeline bubbles. HG-PIPE further introduces careful approximations to implement both linear and non-linear operators with abundant Lookup Tables (LUTs), thus alleviating resource constraints. With a VCK190 FPGA, HG-PIPE realizes end-to-end ViT acceleration on a single device and achieves 7118 images/s, which is 2.81× faster than a V100 GPU.

KEYWORDS

ViT, FPGA, pipeline architecture, hybrid-grained pipeline

1 INTRODUCTION

Recent years have witnessed the wide adoption of Transformer models in the field of computer vision (CV)[6, 20, 31, 1, 32, 36]. While Vision Transformers (ViTs) achieve state-of-the-art (SOTA) performance compared to convolutional neural networks (CNNs), they suffer from a drastic increase in parameters and computation, which calls for more efficient acceleration.

FPGA-based ViT accelerators can be categorized into two architectural paradigms, i.e., temporal architecture [5, 4, 19, 37, 12] and pipelined architecture [42, 28, 39, 49]. Temporal architectures build general processing engines (PEs) for different operators and reuse the PEs temporally [4, 12]. Although temporal architectures simplify the design, they suffer from extensive off-chip memory access[24] and utilization problems. As shown in Figure 1, the temporal architecture (labeled "GeMM") is limited by bandwidth and can only achieve 1.1 TOP/s in our estimation. Pipelined architectures, in contrast, instantiate distinct PEs specialized for different operators and directly stream activations among PEs to reduce off-chip memory access [42, 26, 48, 49].

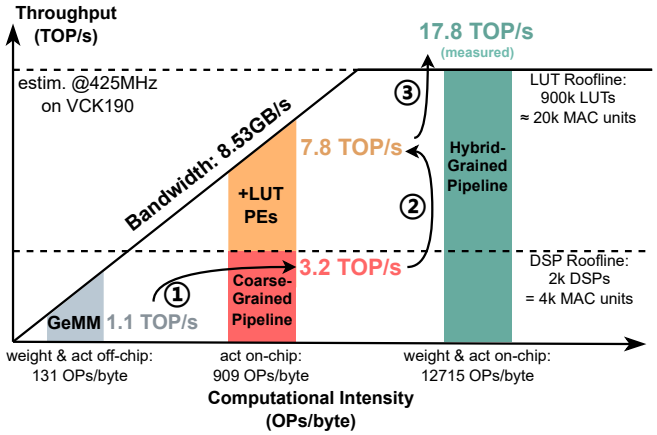


Figure 1: Roofline model for FPGA-based ViT acceleration.

However, to realize the efficiency promise of pipelined architectures faces major challenges. On the one hand, the multi-head attention (MHA) operation in ViTs involves computing the correlation of each token with all other tokens in an image. This introduces global computation dependency and causes either pipeline bubbles [7] or high buffer cost [42]. On the other hand, pipelined architectures compute multiple layers simultaneously and natively require more hardware resources, e.g., digital signal processing (DSP) blocks. The complex non-linear functions in ViT, including GeLU, Softmax, LayerNorm, etc, also require high-precision computation and DSP usage[33, 25, 47]. As shown in Figure 1, only 3.2 TOP/s can be achieved for a coarse-grained pipeline design due to the DSP limitation. If lookup tables (LUTs) are also utilized to construct the PEs, the roofline can be improved, but the design will be limited by bandwidth again and can only achieve 7.8 TOP/s.

To address the challenges, we introduce HG-PIPE[1]in this paper. It combines the features of fine-grained and coarse-grained pipelines to eliminate the bubbles as well as reduce buffer costs. HG-PIPE keeps as many weights on-chip as possible to maximize the computational intensity. It further introduces careful approximations for activation functions to enable LUT-based processing

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICCAD '24, October 27–31, 2024, New York, NY, USA
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1077-3/24/10
https://doi.org/10.1145/3676536.3676681

[1]Our code is available on https://github.com/hguq/HG-PIPE.git

to combat the DSP resource limits. As shown in Figure 1, HG-PIPE breaks through both the DSP roofline and bandwidth limitations, achieving a throughput of 17.8 TOP/s. The contribution of HG-PIPE can be summarized as follows:

- HG-PIPE features a hybrid-grained pipelined architecture to simultaneously achieve low off-chip memory access, low buffer requirements, and negligible pipeline bubbles.
- HG-PIPE leverages low bit-width quantization and introduces careful approximations for activation functions. The abundant LUT resources are utilized to process both linear and non-linear operators, achieving a higher roofline.
- HG-PIPE demonstrates 2.72× better throughput and 2.46× better resource efficiency over prior-art accelerators on a ZCU102 FPGA. It realizes end-to-end ViT acceleration on a single VCK190 FPGA and achieves 7118 images/s.

2 BACKGROUND

2.1 Non-linear Functions in Transformers

ViTs involve various non-linear functions. Different from linear functions, it is challenging to implement them efficiently on FPGAs.

GeLU is applied in the MLP block and can be computed as :

$$\mathrm{GeLU}(x) = \frac{x}{2}\left(1 + \mathrm{erf}\left(\frac{x}{\sqrt{2}}\right)\right), \text{where } \mathrm{erf}(x) = \frac{2}{\sqrt{\pi}}\int_0^x \exp(-t^2)\mathrm{d}t \tag{1}$$

LayerNorm, as shown in Eq. 2, is applied in both the MHA block and the MLP block. The division and the square root operations are fused as the "Rsqrt" operator for hardware efficiency.

$$\mathrm{LayerNorm}(\mathbf{x}) = \frac{\mathbf{x} - \mathrm{E}[\mathbf{x}]}{\sqrt{\mathrm{Var}(\mathbf{x})}} = (\mathbf{x} - \mathrm{E}[\mathbf{x}]) \cdot \mathrm{Rsqrt}(\mathrm{Var}(\mathbf{x})) \tag{2}$$

Softmax, as shown in Eq. 3, is applied in the MHA block to normalize the attention scores. In Softmax, the exponential function (Exp) and the reciprocal function (Recip) are required.

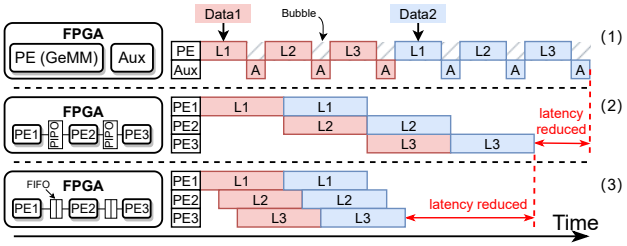
$$\mathrm{Softmax}(\mathbf{x}) = \frac{\mathrm{Exp}(\mathbf{x} - \mathbf{x}_{\max})}{\sum \mathrm{Exp}(\mathbf{x} - \mathbf{x}_{\max})} \tag{3}$$

We also regard the **ReQuant** operator in the quantized networks as a non-linear function. ReQuant can be computed as Eq. 4, where α_{int} and S_{fixed} denote the integer zero point and the fixed-point scaling factor, respectively. ReQuant requires high-precision multiplication and therefore consumes DSPs.

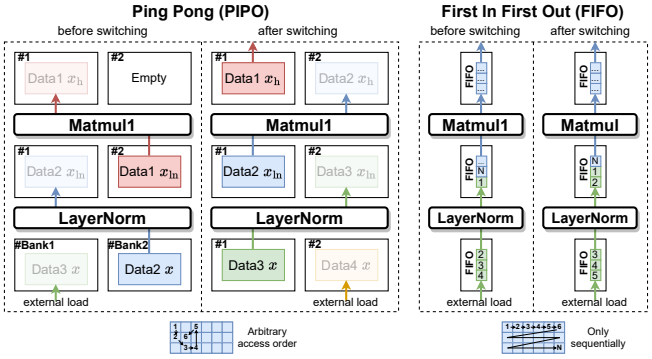
$$\mathrm{ReQuant}(x) = \mathrm{clamp}\left(\lceil (x - \alpha_{\text{int}}) \cdot S_{\text{fixed}} \rfloor, Q_{\min}, Q_{\max}\right) \tag{4}$$

To implement these non-linear functions, three methods have been developed by prior works:

- *Floating point implementation* employs 32-bit or 16-bit floating point computation. Despite its simplicity and precision, it demands substantial DSPs and LUTs.
- *Fixed-point polynomial approximation* uses low-order polynomials to implement non-linear functions within specified ranges[14]. This method is a compromise between computational complexity and accuracy.
- *Lookup table method* involves discretizing the function input range and recording the output. While it reduces DSP usage, it usually requires Block RAMs (BRAMs) for accurate sampling[34].



(a) The difference between pipeline paradigms: (1) temporal architecture, (2) coarse-grained pipeline architecture, (3) fine-grained pipeline architecture. "Aux" is short for auxiliary ops(non-linear functions and off-chip memory access).



(b) The difference between PIPO and FIFO.

	No pipeline (GeMM)	Coarse-grained pipeline	Fine-grained pipeline	Hybrid-Grained Pipeline
Representative Work	[4]	[28]	[46]	Ours
Buffer Type	Global Buffer	PIPO	FIFO	Buffer + FIFO
Buffer Cost	Small	Large	Small	Mid
Data Access Order	Any order	Any order	Sequentially	Any order
Data Access Times	Multiple	Multiple	Only Once	Multiple
ViT Compatibility	✔	✔	✘	✔
Throughput	Low	High	High	High
Latency	High	Mid	Low	Low

(c) Qualitative comparison between different paradigms.

Figure 2: Compare coarse-grained and fine-grained pipeline.

2.2 FPGA-based ViT Acceleration

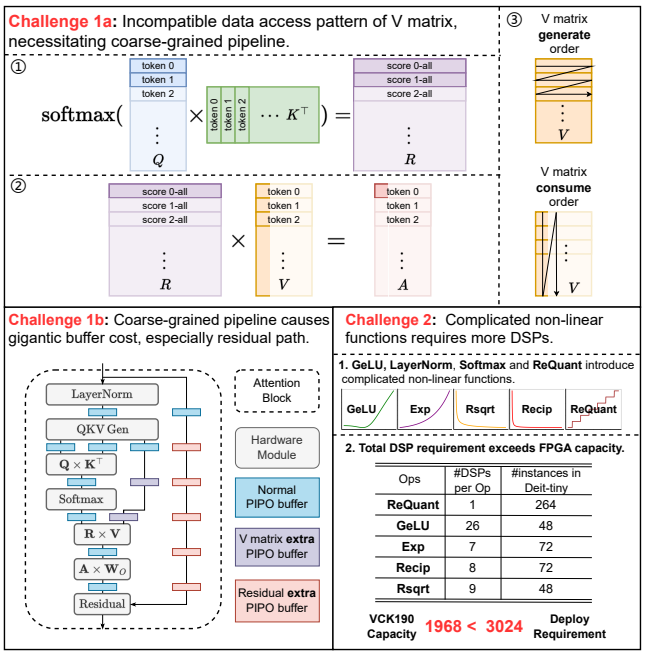
The FPGA-based ViT accelerators can be categorized into two architectures[3]: temporal architecture and pipelined architecture. As shown in Figure 2(a), temporal architectures leverage unified PEs to process different layers. Such PE is typically dedicated to performing General Matrix Multiplication, commonly referred to as GeMM. Most existing FPGA-based ViT accelerators belong to the category [4, 5, 10, 12, 11, 15, 19, 22, 23, 29, 33, 35, 37, 40, 44, 45]. Efficient systolic arrays [16] or sparsity-aware PEs [5, 45] enable temporal architectures to accelerate linear layers effectively. However, these architectures often necessitate frequent off-chip memory access for intermediate results and tend to underutilize resources due to a lack of concurrent multi-operator execution.

Pipelined architectures, as illustrated in Figure 2(a), optimize layer processing by customizing PEs for varied operators across layers [42, 7], enhancing resource utilization and minimizing off-chip

memory costs through inter-PE data transfer [42, 39, 8]. Coarse-grained pipeline processes entire tensors for different operators [21, 18, 43, 8, 17, 28, 42]. On the contrary, fine-grained pipeline tiles activation tensors for sub-tensor processing[26, 48]. A coarse-grained pipeline uses Ping Pong (PIPO) buffers, requiring double the tensor's memory size, while a fine-grained pipeline employs First In First Out (FIFO) buffers with tile-level granularity. The behavior of PIPO and FIFO are depicted in Figure 2(b). Fine-grained pipelined architectures are predominantly utilized in CNN acceleration, providing high hardware utilization and low buffer costs [9, 48, 26, 30], whereas coarse-grained architectures are preferred for ViT acceleration [21, 18, 43, 8, 17, 28, 42] for data access ability. A comparative analysis of these architectures is presented in Table 2(c).

3 CHALLENGES

Although ViT has outstanding performance, its complexity brings challenges for FPGA accelerator design as depicted in Figure 3. These challenges motivated us to propose HG-PIPE.



Ops	#DSPs per Op	#instances in Deit-tiny
ReQuant	1	264
GeLU	26	48
Exp	7	72
Recip	8	72
Rsqrt	9	48

Figure 3: The challenges of ViT hardware acceleration.

Challenge 1: Dataflow Design Dilemma. The self-attention mechanism in ViTs introduces global computation dependencies, leading to challenges in data locality and dataflow design. As illustrated in Figure 3 (1a), the transpose operation within the self-attention block breaks the data access pattern for the **V** matrix, hampering the continuity needed for fine-grained pipelines and necessitating coarse-grained pipelines. However, as shown in Figure 3 (1b), the coarse-grained approach causes gigantic activation buffering costs, exacerbated by residual connections that require multiple Ping Pong buffers to avoid deadlock. Specifically, in the Deit-tiny network, buffering one residual tensor consumes 14 BRAMs. One attention block in a coarse-grained pipeline requires 6 PIPO stages (168 BRAMs) just for the residual path. This extensive buffering demand is impractical for FPGA platforms[42].

Challenge 2: High DSP Usage. The extensive non-linear functions in ViT present challenges for efficient FPGA implementation. In our HLS synthesis experiments, naive floating-point implementations of functions like **Exp**, **Rsqrt**, and **Recip** are DSP-intensive, consuming 7, 8, and 9 DSPs respectively. The **GeLU** function is even more DSP-intensive, requiring 26 DSPs. The **ReQuant** function additionally uses 1 DSP. In the estimation, implementing these non-linear functions in a Deit-tiny model requires 3024 DSPs, exceeding the DSP capacity of a VCK190 FPGA. Therefore, reducing DSP usage in ViT accelerators is crucial for FPGA implementation.

4 MAIN METHODS

4.1 Overview of HG-PIPE

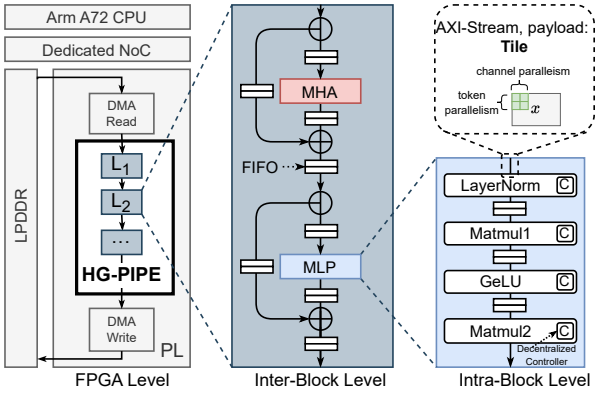


Figure 4: Overview of the HG-PIPE accelerator design.

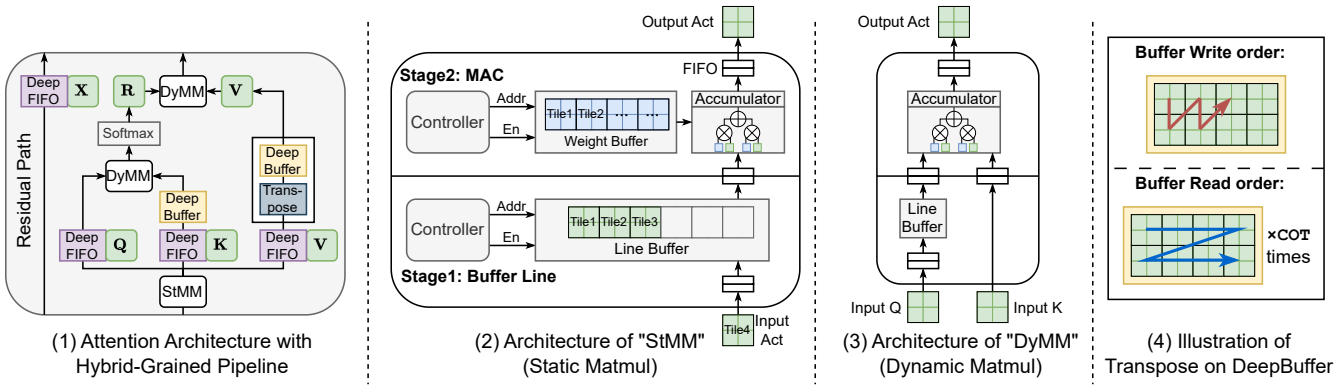
The overview of the HG-PIPE accelerator is depicted in Figure 4. It is integrated into an FPGA System-on-Chip (SoC) framework, with multiple peripheral components, including external memory, host system, and network-on-chip (NoC).

HG-PIPE stands out with its fully pipelined architecture across all layers, eliminating external memory access for intermediate activations or weights. It directly processes incoming data from the Direct Memory Access (DMA) module, channeling the final output back to external memory. HG-PIPE utilizes dedicated modules for each component, avoiding time division multiplexing.

To achieve sufficient throughput, HG-PIPE incorporates over 20,000 MAC units. Managing such a vast array of MAC units typically complicates control logic. Centralized control often leads to extended routing nets that hamper timing. To mitigate this, HG-PIPE adopts an asynchronous, decentralized pipeline strategy, where each stage is controlled by its own FSM. With handshakes on the AXI-Stream interface, modules are completely decoupled. The design incorporates FIFOs within these connections to cover data generation fluctuations, avoiding deadlocks in the data flow. As illustrated, data is transferred in the form of tiled tensors.

4.2 Hybrid-Grained Pipeline

To address the challenges outlined in Sec. 3, we developed a hybrid-grained pipeline to effectively manage computing and buffering



(1) Attention Architecture with Hybrid-Grained Pipeline

(2) Architecture of "StMM" (Static Matmul)

(3) Architecture of "DyMM" (Dynamic Matmul)

(4) Illustration of Transpose on DeepBuffer

Figure 5: The dataflow design of the MHA module.

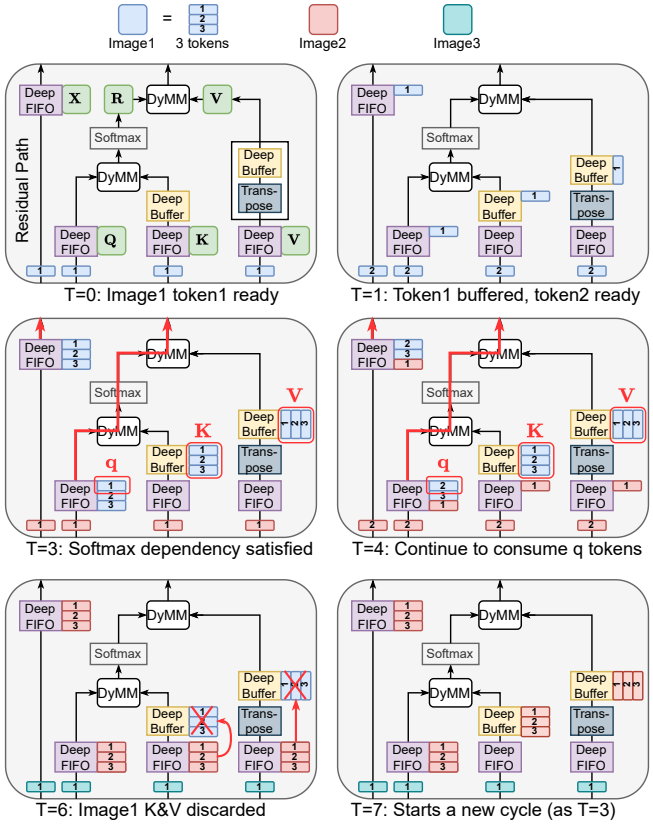


Figure 6: The time diagram of the hybrid-grained pipeline.

across different granularities. Despite the dependencies across tokens introduced by the Softmax operator and transposed matrix multiplication, the attention mechanism retains substantial locality in other segments. Notably, the attention block comprises four branches: the residual branch, and Q/K/V branches. The Q branch exhibits good data locality, as illustrated in Figure 3 (1a), where generating one **A** token requires one **Q** token and the entirety of **K** and **V** tokens. In particular, the V matrix requires row-wise access, differing from token-wise access.

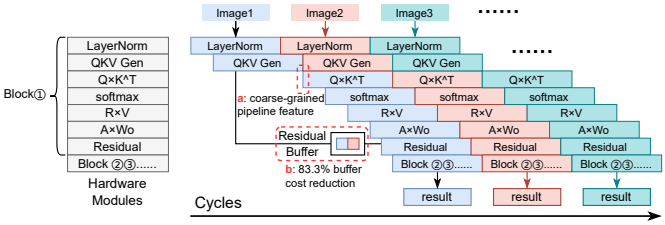
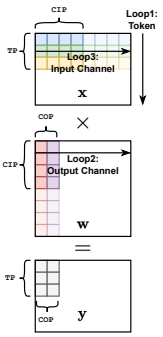


Figure 7: The hardware view of the hybrid-grained pipeline.

To solve the granularity problem, we implemented deep buffers in the K and V branches and introduced a "Transpose Module" in the V branch to re-order access, aligning it with the Q matrix's fine-grained pipeline, depicted in Figure 5 (1) and (4). The buffer is deep enough to hold the entire **K** or **V** tensor. The content of K and V buffers will be read COT times to generate all output channels, which will be explained in Sec. 4.3. Figure 5 (2) and (3) illustrate the architecture of "StMM" and "DyMM". "StMM" represents matrix multiplications (MMs) with static weights (e.g., QKV generation and MLP MMs), and "DyMM" represents MMs with dynamic weights (e.g., $\mathbf{Q} \times \mathbf{K}^\top$ and $\mathbf{R} \times \mathbf{V}$). They both consist of two stages, with the first stage buffering all input channels and the second stage performing MAC operations. The difference is that "DyMM" streams the dynamic weight from the previous stage buffer while "StMM" freezes the weights as ROMs. The deep buffers guarantee coarse-grained access for dynamic weights (i.e., **K** and **V**).

However, deep buffers alone were insufficient since other branches would be halted in the execution. Thus, we incorporated deep FIFOs into all four branches to maintain continuous execution. We carried out simulation experiments to identify the shallowest depth that avoids deadlocks, and the typical depth of deep FIFOs is 512. The mechanism of deep FIFOs and deep buffers is illustrated in Figure 6. The process is simplified and focuses on key stages for clarity, with each image containing 3 tokens. At T=0, the pipeline is initialized with the first token of Image1 (blue) queued in the prior stage. The residual and Q tokens are stored in deep FIFOs, while K and V tokens are placed directly into deep buffers, with V tokens being transposed first. By T=3, all K and V tokens of Image1 are buffered, therefore the data dependencies for attention are met, allowing the first Q token to be consumed and the first A token



Notation	Explanation
T	Sequence length
CI	Embedding dimension (input channels)
CO	Hidden layer dimension (output channels)
TP	Parallelism on Sequence length
CIP	Parallelism on input channels
COP	Parallelism on output channels
TT	Trip count on T direction; TT = T/TP
CIT	Trip count on CI direction; CIT = CI/CIP
COT	Trip count on CO direction; COT = CO/COP
$\mathbf{x}, \mathbf{w}, \mathbf{y}$	Input tokens, weight, output tokens, $\mathbf{x}$.shape = (T, CI), $\mathbf{w}$.shape = (CO, CI), $\mathbf{y}$.shape = (T, CO)
$DW_{\mathbf{x}}, DW_{\mathbf{w}}$	Data width of $\mathbf{x}$, $\mathbf{w}$
B_{width}, B_{depth}	BRAM bank width, depth

Figure 8: The notations of tiled matrix multiplication.

to be generated (as highlighted by the red arrow). As Image1's Q tokens are consumed, tokens from Image2 (red) start to load. By T=6, after all Q tokens of Image1 are processed, the K and V buffers refresh for Image2, initiating a new cycle at T=7.

Figure 6 shows token-level dataflow, while our design operates at a more fine-grained sub-token granularity, detailed in Sec. 4.3. Figure 7 provides a hardware perspective, demonstrating overlaps of operators and hybrid-grained features. "QKV Gen" module initiates before "LayerNorm" completes as a fine-grained pipeline, and as highlighted (a), "$Q \times K^\top$" cannot start early due to dependencies as a coarse-grained pipeline. The residual buffer cost is significantly reduced by 83.3% compared to traditional PIPO implementation (b).

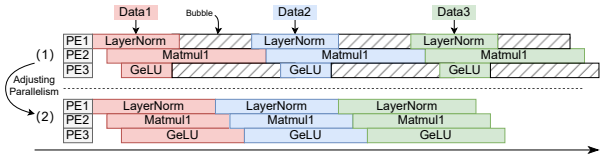
4.3 Parallelism Design

Matrix Multiplication (MM) is the primary operator used in ViT computations. We implement tiled MM using an Output Stationary (OS) dataflow to minimize partial sum storage costs, as illustrated in Figure 8. MM involves three nested loops: Token, Output Channel, and Input Channel. All loops are tiled to enhance data locality. For clarity, notations are detailed in the Table. We meticulously design the parallelism of all the modules for two reasons: pipeline balance and BRAM utilization efficiency.[2]

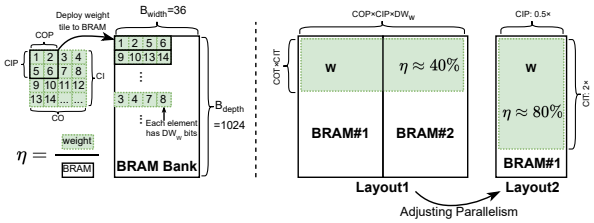
4.3.1 Pipeline Balance. The pipeline balance is essential for the accelerator's performance. The cycle of the whole pipeline is the maximum of the cycles of all the pipeline stages. As shown in Figure 9a(1), if the cycles are not balanced, the bubbles will be generated. Therefore, by allocating more computing resources to the Matmul1 module, the pipeline can be balanced in Figure 9a(2). By adjusting the parallelism, we try to make the cycles of the pipeline stages balanced. The calculation of cycles is detailed in Table 1 footnotes.

4.3.2 BRAM Utilization Efficiency. BRAM is a scarce resource on FPGA for AI applications. It is also a critical resource for our design since HG-PIPE needs to freeze as much of the weights on-chip as possible to reduce external memory access. The parallelism design can directly decide the memory layout of the weights in BRAM (as shown in Figure 9b), and improper memory layout will lead to BRAM waste. In Layout1, the weight $\mathbf{w}$ requires two BRAMs to accommodate it. However, by scaling CIP to half of its original value in Layout2, there will only one BRAM be required. As illustrated in

[2]We didn't apply an automatic process for generating parallelism hyperparameters, since in transformers, each layer has the same structure and shape, deriving a small design space. Hand-crafted design is feasible and good enough.



(a) Eliminating imbalance-induced bubble in a fine-grained pipeline.



(b) Adjust the parallelism design to improve BRAM utilization.

Figure 9: Adjusting parallelism to improve efficiency.

Module Name		T/TP=TT	CI/CIP=CIT	CO/COP=COT	MOPs[1]	P[2]	Cycles[3]	η[4]
MHA	LayerNorm	196/2=98	192/1=192	-	0.11	2	56448	-
	QKV Gen	196/2=98	192/6=32	64/4=16	2.41	48	50176	100%
	QK MatMul	196/2=98	64/4=16	196/7=28	2.46	56	43904	68.1%
	Softmax	196/2=98	196/1=196	-	0.11	2	57624	-
	RV MatMul	196/2=98	196/7=28	64/4=16	2.46	56	43904	68.1%
	Output Proj	196/2=98	192/12=16	192/6=32	7.23	144	50176	100%
	Residual Add	196/2=98	192/1=192	-	0.038	2	18816	-
MLP	LayerNorm	196/2=98	192/1=192	-	0.11	2	56448	-
	MatMul1	196/2=98	192/12=16	768/24=32	28.9	576	50176	100%
	GeLU	196/2=98	192/2=98	-	0.15	4	37632	-
	MatMul2	196/2=98	768/24=32	192/12=16	28.9	576	50176	100%

[1] MOPs: Million Operations. MOPs = $\mathtt{T} \times \mathtt{CI} \times \mathtt{CO}$.

[2] P: Parallelism in total. For Matrix Multiplication, P is the number of parallel MAC units. For other modules, P is the number of parallel elementwise or reduction units. $\mathtt{P} = \mathtt{TP} \times \mathtt{CIP} \times \mathtt{COP}$.

[3] Cycles: The number of cycles required by the module for one inference. For **LayerNorm** and **Softmax** modules, three passes are needed, therefore they require 3 times cycles. Cycle $= \mathtt{TT} \times \mathtt{CIT} \times \mathtt{COT}$. For an accelerator with multiple stages, the cycle of the whole accelerator is the maximum of the cycles of all the stages: $\text{Cycle}_{\text{accelerator}} = \max(\text{Cycle}_{\text{stage1}}, \text{Cycle}_{\text{stage2}}, \cdots, \text{Cycle}_{\text{stageN}})$.

[4] η: BRAM Efficiency. $\#\text{BRAM} = \left\lceil \frac{DW_{\mathbf{w}} \cdot \mathtt{CIP} \cdot \mathtt{COP}}{B_{width}} \right\rceil \cdot \left\lceil \frac{\mathtt{CIT} \cdot \mathtt{COT}}{B_{depth}} \right\rceil$, and $\eta = \frac{DW_{\mathbf{w}} \cdot \mathtt{CI} \cdot \mathtt{CO}}{\#\text{BRAM} \cdot B_{width} \cdot B_{depth}}$.

Table 1: The parallelism design result on Deit-tiny.

Table 1 footnotes, we can calculate the number of BRAMs required by one MM module (denoted as #BRAM), and the corresponding BRAM utilization efficiency (denoted as η).

4.3.3 Parallelism Design Results. The parallelism design results are shown in Table 1, in which we tried to achieve the two goals at the same time. As demonstrated, most modules have closely matched cycles. To save DSP resources, we choose the non-linear operators to be the cycle bottleneck (57624 of the Softmax module). For the Residual Add module, we keep TP = 2 to simplify the design, leading to smaller cycle values. Although bubbles exist in Residual, this is not a big waste since it only requires 0.038 MOPs. For the BRAM utilization efficiency, we achieved 100% efficiency for all the modules that employ static weight (QKV Gen, Output Proj in MHA block and MatMul1, MatMul2 in MLP block).

4.4 Efficient and Accurate LUT-based Processing

ViTs incorporate complex non-linear functions, previously discussed in Sec. 2.1 and Sec. 3. Accurately computing these functions is crucial for maintaining model performance, but full-precision

floating-point calculations exceed FPGA DSP capacity. In this section, we provide optimization and approximation strategies that enhance hardware friendliness while preserving accuracy, allowing non-linear functions to be effectively implemented using LUTs.

4.4.1 LUT-based MAC unit. The implementation of multiplication operations using LUTs is a technique in FPGA-based computations. Consider a scenario where operands of multiplication are quantized to 3 bits. In this case, the multiplication operation can be decomposed into six boolean functions, with each function consuming 6 bits to produce a single bit of the multiplication. As a result, only 6 LUT-6 are required. The adoption of LUTs for MAC units can significantly enhance the computational capabilities of an FPGA, effectively raising its computation roofline. This approach has been successfully utilized in several accelerator works [19, 26, 41, 48]. We have incorporated this technique in our design as well.

4.4.2 Power-of-Two Index Approximation. In the LUT method, the index computation is essential. The process is similar to quantization in Eq. 4: discretizing continuously distributed input values. In the traditional table method, the index is computed as Eq. 5.

$$\text{index} = \left\lceil (\text{data} - \alpha) \times \frac{2^n - 1}{\beta - \alpha} \right\rfloor \tag{5}$$

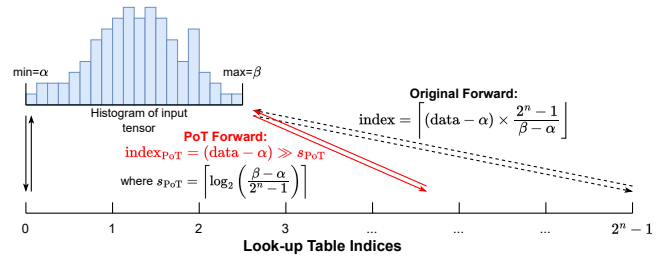
in which (α, β) is the data range of input tensor, and n is the address width of the table. However, this method will require a DSP for the multiplication. This is contradictory to the motivation of the table method: reduce DSP usage. The similarity between index computation and quantization inspired us to introduce a quantization technique into our LUT methods: Power-of-Two Quantization [27, 2, 48]. This method estimates the scaling factor with its nearest Power-of-Two (PoT) value, therefore simplifying the high-precision multiplication to a bit shifting. The estimation is shown in Eq. 6.

$$\text{index}_{\text{PoT}} = (\text{data} - \alpha) \gg s_{\text{PoT}}, \quad \text{where } s_{\text{PoT}} = \left\lceil \log_2 \left(\frac{\beta - \alpha}{2^n - 1} \right) \right\rceil \tag{6}$$

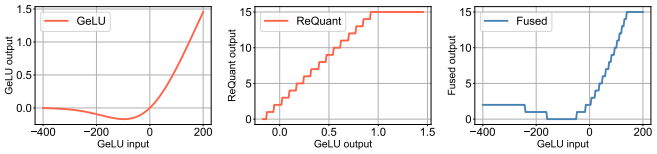
Our estimation is a bit different from vanilla PoT Quantization. We apply a ceiling instead of rounding to avoid index overflowing. The whole process is shown in Figure 10a. The blue bars show the data distribution of the input tensor. Taking the min and max values, the normal scaling factor can be calculated, and the black dashed arrows show the normal index mapping, aligning the boundaries and mapping β to the highest index $(2^n - 1)$. On the contrary, as shown by the red arrows, PoT estimation will not guarantee boundary alignment, but it will make sure the scaling happens as a static bit shifting.

4.4.3 GeLU-ReQuant Operator Fusion. In a quantized network, quantizers are inserted before all MMs to improve MAC efficiency. Fusing quantizers with preceding operators simplifies the logic and reduces LUT consumption. The fusion involves sampling a combined transfer curve, depicted in Figure 10b. The left sub-figure represents the GeLU function, the middle one the ReQuant function with 4-bit precision, and the right one the resulting fused curve. The sampling will happen on the combined curve.

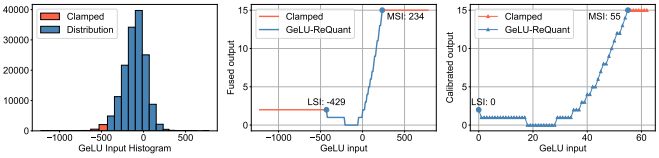
4.4.4 Implement ReQuant as Table. Besides GeLU fusions, numerous ReQuant operators cannot be fused and also consume DSP



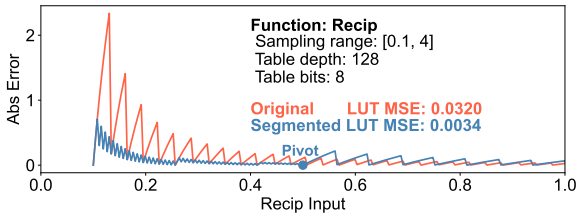
(a) The computation of index and the Power-of-Two approximation method.



(b) The fusion of GeLU operator and ReQuant operator.



(c) The joint table range calibration for GeLU and ReQuant.



(d) The segmented table for the Recip operator with high dynamic range. The segmentation pivot is annotated on the curve.

Figure 10: LUT optimization techniques.

resources. By treating quantizers as non-linear functions, we can apply Power-of-Two index approximation to eliminate DSP usage as well. Employing low-precision quantization significantly reduces LUT cost. Our experiments show that a 64-entry ReQuant table sufficiently preserves accuracy.

4.4.5 Joint Table Range Calibration. In the implementation result of the ReQuant table and GeLU-ReQuant table, we discovered that there are many repeated entries generated from the clamping behavior (Eq. 4) on the two ends, causing wastes of representative ability. This is demonstrated in the middle sub-figure of Figure 10c, in which the repeated entries on the two ends are colored orange. We introduce Joint Table Range Calibration to reduce redundancy in the table. This method aligns the input data distribution with the table contents to optimize implementation. It iteratively identifies the Least Significant Index (LSI) and Most Significant Index (MSI) to recalculate the data range and updates the table until the data

range stabilizes. In Figure 10c, the left sub-figure shows clamped data in orange. After calibration, the LSI is mapped to 0, and MSI is mapped near the right boundary. There will still be a few remaining repeated entries on the right side due to PoT approximation.

4.4.6 Segmented Table for High Dynamic Range Recip. The Recip function in the Softmax module exhibits a high dynamic range in our statistic experiments. To accurately sample the function results on the sheer range between 0 to 1, the table needs to be very large. Initially, storing the reciprocal function would have needed an entire BRAM bank (depth=1024, width=36) to maintain accuracy. To minimize BRAM usage, we exploited the function's inherent properties and segmented it into two parts, each owning an independent scaling factor. We empirically divide the input range at the first 1/8 for the steep part and the remainder for the flat. This approach is visualized in Figure 10d. The orange line is the abs error of the original LUT implementation of Recip, and the blue one is the segmented implementation with the segmentation pivot annotated to it. With more entries between 0 and 1, the sampling is more accurate, reducing Mean Squared Error (MSE) from 0.032 to 0.0034.

4.4.7 Inversed Exponential Table. In our experiment, we observed that the PoT approximation on Exp will cause huge accuracy degradation. The possible explanation is that in the calculation of Softmax, each element is subtracted by the maximum value in its group to maintain numerical stability. Therefore, the max value of the input range is anchored to 0. However, the approximation described in Sec. 4.4.2 takes the min value as the zero point. The moving of the anchor point may cause an inaccurate approximation of the sensitive anchor values. The solution is simple: we make β the zero point and modify the index computation from Eq. 6 to Eq. 7:

$$\text{index}_{\text{PoT}} = (\beta - \text{data}) \gg s_{\text{PoT}}, \quad \text{where } s_{\text{PoT}} = \left\lceil \log_2 \left(\frac{\beta - \alpha}{2^n - 1} \right) \right\rceil \tag{7}$$

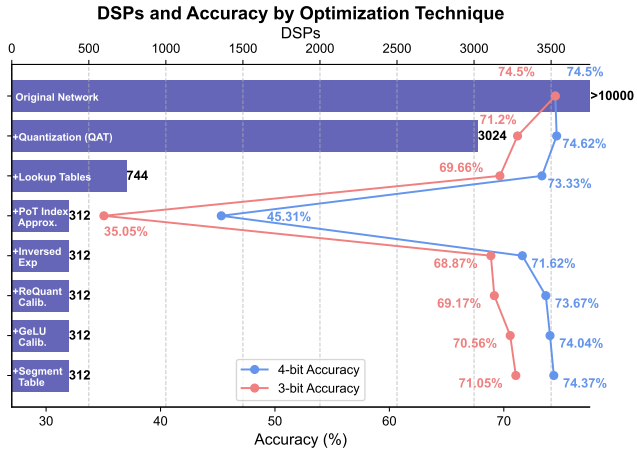
5 EXPERIMENTS

5.1 Experiment Setup

To evaluate our design, we selected the Deit-tiny model and Deit-small model, following our baselines AutoViTAcc [19], HeatViT [5] and SSR[49]. We tested on two FPGA platforms: ZCU102 and VCK190. The ZCU102 allows direct comparisons with prior works, while the VCK190 supports full deployment of the whole network. Throughput was measured using the PYNQ framework, and power consumption was assessed with Xilinx's BEAM tool.

5.2 Timing Diagram of the Pipeline

We simulated our design to generate a timing diagram of blocks, shown in Figure 12. The accelerator sequentially loads input tensor tiles. As Image1's loading completes, Image2's begins, indicating overlapped execution. The MHA block exhibits coarse-grained buffering, causing a slight delay in outputting the first tile. For the third image, the stable cycles measured was 57,624 cycles as expected, validating the hybrid-grained pipeline's effectiveness. The trace also reveals that the total processing time for Image1 is 824,843



(a) DSP usage and accuracy with all the optimization applied step by step

Ablation	Deit-tiny 3bit	Deit-tiny 4bit
w/o Inverted Exp	28.80% (**-42.25%**)	48.87% (**-25.50%**)
w/o ReQuant Calib.	70.56% (**-0.49%**)	74.08% (**-0.29%**)
w/o GeLU Calib.	69.49% (**-1.56%**)	72.44% (**-1.93%**)
w/o Segmented Recip	70.57% (**-0.48%**)	74.04% (**-0.33%**)

(b) Ablation study experiment results.

Non-linear Functions	Table depth	Table bits	LUT-6 Cost Reduction	DSP Cost Reduction
Exp	64	8	945 → 50	7 → 0
GeLU	64	3	1650 → 43	26 → 0
Recip	64*2	8	196 → 72	8 → 0
Rsqrt	64	12	425 → 48	9 → 0
ReQuant	64	3	0 → 3	1 → 0

(c) Resource consumption reduction with LUT methods.

Figure 11: Experiment results on LUT optimizations.

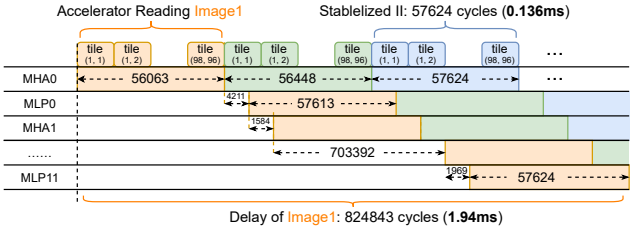


Figure 12: The timing diagram in HG-PIPE of each block.

cycles or 1.94ms. Using the stable cycles, the computed latency is 0.136 ms, equating to an ideal frame rate of 7353 images/s.

5.3 Comparison with Related Works

We benchmarked HG-PIPE against leading works, as detailed in Table 2. Our hybrid-grained pipeline design demonstrated significant improvements in throughput, resource efficiency, and power efficiency. On the ZCU102 platform, compared to AutoViTAcc, HG-PIPE achieves a LUT efficiency of 18.55 GOPs/kLUT, which is 2.52 times higher under the same 4-bit quantization and platform conditions. On the VCK190 platform, HG-PIPE achieved a throughput of

	Deit GPU Baseline [38]	TCAS-I 2023 [12]	**AutoViTAcc** FPL 2022 [19]	**HeatViT** HPCA 2023 [5]	**SSR** FPGA 2024 [49]	**HG-PIPE** This work 2024			
Paradigm	GPU	GeMM	GeMM	GeMM	Coarse-Grained Pipeline	**Hybrid-Grained Pipeline**			
FPGA/GPU	V100 GPU	ZCU102	ZCU102	ZCU102	VCK190	ZCU102[3]	VCK190	VCK190	VCK190
Frequency	1455MHz	300MHz	150MHz	150MHz	PL:250MHz, AIE:1GHz	375MHz	425MHz	425MHz	350MHz
Network	Deit-tiny	ViT-tiny	Deit-small	Deit-tiny	Deit-tiny	Deit-tiny	Deit-tiny	Deit-tiny	Deit-small
OPs/inf	2.5G	2.5G	9.2G	2.5G	2.5G	2.5G	2.5G	2.5G	9.2G
Params	5.5M	5.5M	22M	5.5M	5.5M	5.5M	5.5M	5.5M	22M
Precision	fp32	A8W8	A4W4+A4W3	A8W8	A8W8	A4W4	A4W4	A3W3	A3W3
Accuracy	74.5%	73.00%	77.94%	72.20%	-	74.37%	74.37%	71.05%	-[6]
FPS	2529	245	155.8	183.4	4545	1579	3629	**7118**	1490
GOPs	6322.5	762.7	1418.4	366.8	11362.5	3947.5	9072.5	**17795**	13708
LUTs[1]	-	114k	193k	137.6k	619k	212.7k	514k	669k	869k
DSPs	-	1268	1549	1968	14405[5]	78	156	312	312
BRAMs[2]	-	648	-	355.5	1456	324.5	1284	1006.5 [4]	2748
Power	-	29.6W	10.34W	9.45W	46W	21.9W	43.4W	46.7W	48.1W
GOPs/kLUT	-	6.69	7.35	2.66	18.35	18.55	17.65	**26.60**	15.77
GOPs/DSP[7]	-	0.157	0.187	0.058	0.336	0.587	0.559	**0.839**	0.499
GOPs/W	-	25.76	137.17	38.80	246.15	180.25	209.04	**381.0**	284.9

[1,2] **LUTs**: LUT-6 for AMD FPGAs. **BRAMs**: BRAM-36k for AMD FPGAs.
[3] The ZCU102 board is not capable of freezing all the layers of Deit-tiny, therefore we divide the network into 4 parts.
[4] We allocated 718.5 BRAMs and 36 URAMs to this setup. For the sake of comparison, we normalized the utilization of URAMs to BRAMs, considering 1 URAM equivalent to 8 BRAMs.
[5] SSR[49] utilized 394 AI Engine (AIE) to perform the matrix multiplication, we normalized the utilization of AIE to DSPs, considering 1 AIE equivalent to 32 DSPs.
[6] Currently we do not have Quantization Aware Training (QAT) weight results for Deit-small or bigger models.
[7] **Normalized GOPs/DSP**: Since different works adopt different LUT/DSP preferences, we normalize them to be DSP-only for a fair comparison, considering 1 DSP = 32 LUTs. This is a very conservative estimation since 1 DSP is typically more capable than 32 LUTs, and likely underestimates our true efficiency since we predominantly use LUTs. Nonetheless, our work still demonstrates superior efficiency compared to other works.

Table 2: Comparison with prior art works.

7118 images/s and 17.8 TOPs/s, which is 96.8% of the ideal 7353 images/s throughput. Compared to the V100 GPU (2529 FPS), HG-PIPE outperforms it by 2.81 times. For DSP usage, HG-PIPE drastically reduced consumption to just 312 DSPs through effective LUT-based optimizations. Moreover, HG-PIPE also demonstrates great power efficiency. On the same VCK190 platform, HG-PIPE achieves 381.0 GOPs/W, outperforming prior-art work SSR (246.15 GOPs/W).

5.4 LUT Optimization Techniques

We evaluated the impact of our LUT optimization techniques in reducing DSP usage and maintaining network accuracy in Figure 11a. Initially, the network had 74.5% accuracy with a prohibitive DSP cost of 14304 [38]. Implementing advanced quantization techniques [13] allowed us to implement MAC units with LUTs, reducing DSP usage from 14304 to 3024. Further applying Power-of-Two LUTs for non-linear functions cut DSP usage to 312, albeit with a significant accuracy loss. The crucial Inverted Exp Table technique significantly restored accuracy, with subsequent optimizations gradually recovering it while keeping DSP usage constant. We also conducted an ablation study to pinpoint the effects of optimizations detailed in Figure 11b. Also, Figure 11c demonstrates the table sizes and the resource reduction effects. As listed, LUT costs of non-linear functions are significantly reduced, while DSP usage is eliminated.

5.5 Test Environment and Device View

Our test environment is demonstrated in Figure 13. The VCK190 board is connected to the host PC via Ethernet. The screen of the host PC displays the BEAM and Jupyter Notebook GUI, highlighting real-time power consumption and frame rate. On the right side, the implementation device view is elaborated with each module annotated. The device view contains 26 neural network blocks, including PatchEmbed, Classification Head, 12 MHA blocks and 12

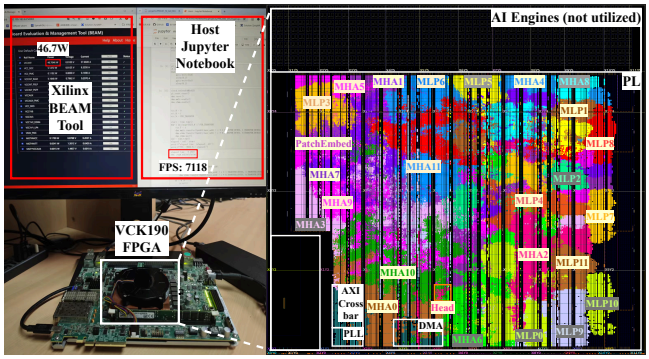


Figure 13: Test environment and device view of HG-PIPE.

MLP blocks. The placement is automatically generated by Vivado, thus appearing in a random order. Other modules such as PS, PLL, DMA and AXI Crossbar are also annotated. Notably, no AI Engine is utilized and all computation is performed on the PL side.

6 CONCLUSION

In this paper, we introduce HG-PIPE, a hybrid-grained pipelined ViT accelerator optimized for FPGAs. By combining the advantages of fine-grained and coarse-grained designs, our approach achieves low latency and high resource efficiency at the same time, reducing the on-chip activation buffering cost by 83.3%. We have implemented optimizations including PoT table index computation and LUT-based ReQuant, which collectively reduce DSP usage by 89.6% without sacrificing accuracy. On ZCU102, HG-PIPE outperforms existing accelerators, achieving 2.72× higher throughput and 2.46× better resource efficiency. On VCK190, HG-PIPE delivers real-time ViT processing at 7118 FPS, equivalent to 17.8 TOP/s.

REFERENCES

[1] Nicolas Carion et al. 2020. End-to-end object detection with transformers. In *European conference on computer vision*. Springer.

[2] Sung-En Chang et al. 2021. Mix and Match: A Novel FPGA-Centric Deep Neural Network Quantization Framework. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. (Feb. 2021), 208–220.

[3] Hongzheng Chen et al. 2024. Understanding the potential of fpga-based spatial acceleration for large language model inference. *ACM Transactions on Reconfigurable Technology and Systems*.

[4] Yonghao Chen et al. 2023. High-frequency systolic array-based transformer accelerator on field programmable gate arrays. *Electronics*, 12, 4.

[5] Peiyan Dong et al. 2023. HeatViT: Hardware-Efficient Adaptive Token Pruning for Vision Transformers. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. (Feb. 2023), 442–455.

[6] Alexey Dosovitskiy et al. 2020. An image is worth 16x16 words: transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*.

[7] Hongxiang Fan et al. 2022. Adaptable Butterfly Accelerator for Attention-based NNs via Hardware and Algorithm Co-design. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. (Oct. 2022), 599–615.

[8] Xiangqu Fu et al. 2023. P3 vit: a cim-based high-utilization architecture with dynamic pruning and two-way ping-pong macro for vision transformer. *IEEE Transactions on Circuits and Systems I: Regular Papers*.

[9] Mathew Hall et al. 2020. From TensorFlow Graphs to LUTs and Wires: Automated Sparse and Physically Aware CNN Hardware Generation. In *2020 International Conference on Field-Programmable Technology (ICFPT)*. (Dec. 2020), 56–65.

[10] Yuntao Han et al. 2023. HPTA: A High Performance Transformer Accelerator Based on FPGA. In *2023 33rd International Conference on Field-Programmable Logic and Applications (FPL)*. (Sept. 2023), 27–33.

[11] Wei Hu et al. 2021. Vis-top: visual transformer overlay processor. *arXiv preprint arXiv:2110.10957*.

[12] Mingqiang Huang et al. 2023. An integer-only and group-vector systolic accelerator for efficiently mapping vision transformer on edge. *IEEE Transactions on Circuits and Systems I: Regular Papers*.

[13] Xijie Huang et al. 2023. Variation-aware vision transformer quantization. *arXiv preprint arXiv:2307.00331*.

[14] Sehoon Kim et al. 2021. I-bert: integer-only bert quantization. In *International conference on machine learning*. PMLR, 5506–5518.

[15] Zhenglun Kong et al. 2023. Peeling the onion: hierarchical reduction of data redundancy for efficient vision transformer training. In *Proceedings of the AAAI Conference on Artificial Intelligence* number 7. Vol. 37, 8360–8368.

[16] Hsiang Tsung Kung et al. 1979. Systolic arrays (for vlsi). In *Sparse Matrix Proceedings 1978*. Vol. 1. Society for industrial and applied mathematics Philadelphia, PA, USA.

[17] Eunji Kwon et al. 2023. Mobile Accelerator Exploiting Sparsity of Multi-Heads, Lines, and Blocks in Transformers in Computer Vision. In *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. (Apr. 2023), 1–6.

[18] Tianyang Li et al. 2023. Unified Accelerator for Attention and Convolution in Inference Based on FPGA. In *2023 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, Monterey, CA, USA, (May 21, 2023), 1–5. ISBN: 978-1-66545-109-3.

[19] Zhengang Li et al. 2022. Auto-ViT-Acc: An FPGA-Aware Automatic Acceleration Framework for Vision Transformer with Mixed-Scheme Quantization. In *2022 32nd International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, Belfast, United Kingdom, (Aug. 2022), 109–116. ISBN: 978-1-66547-390-3.

[20] Ze Liu et al. 2021. Swin transformer: hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, 10012–10022.

[21] Zejian Liu et al. 2021. Hardware Acceleration of Fully Quantized BERT for Efficient Natural Language Processing. In *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. (Feb. 2021), 513–516.

[22] Zhiyang Liu et al. 2023. An efficient fpga-based accelerator for swin transformer. *arXiv preprint arXiv:2308.13922*.

[23] Zizhang Luo et al. 2023. Calabash: Accelerating Attention Using a Systolic Array Chain on FPGAs. In *2023 33rd International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, Gothenburg, Sweden, (Sept. 4, 2023), 242–247. ISBN: 9798350341515.

[24] Linyan Mei et al. 2023. Defines: enabling fast exploration of the depth-first scheduling space for dnn accelerators through analytical modeling. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 570–583.

[25] Zhengyu Mei et al. 2023. Tea-s: a tiny and efficient architecture for plac-based softmax in transformers. *IEEE Transactions on Circuits and Systems II: Express Briefs*.

[26] Jian Meng et al. 2021. FixyFPGA: Efficient FPGA Accelerator for Deep Neural Networks with High Element-Wise Sparsity and without External Memory Access. In *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*. (Aug. 2021), 9–16.

[27] Daisuke Miyashita et al. 2016. Convolutional Neural Networks using Logarithmic Data Representation. (Mar. 16, 2016). arXiv: 1603.01025 [cs]. Retrieved Mar. 13, 2023 from http://arxiv.org/abs/1603.01025. preprint.

[28] Hongwu Peng et al. 2022. A length adaptive algorithm-hardware co-design of transformer on FPGA through sparse attention and dynamic pipelining. In *Proceedings of the 59th ACM/IEEE Design Automation Conference* (DAC '22). Association for Computing Machinery, New York, NY, USA, (Aug. 23, 2022), 1135–1140. ISBN: 978-1-4503-9142-9.

[29] Patrick Plagwitz et al. 2022. TRAC: Compilation-Based Design of Transformer Accelerators for FPGAs. In *2022 32nd International Conference on Field-Programmable Logic and Applications (FPL)*. (Aug. 2022), 17–23.

[30] Jiantao Qiu et al. 2016. Going deeper with embedded fpga platform for convolutional neural network. In *Proceedings of the 2016 ACM/SIGDA international symposium on field-programmable gate arrays*, 26–35.

[31] Joseph Redmon et al. 2016. You only look once: unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 779–788.

[32] Olaf Ronneberger et al. 2015. U-net: convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention–MICCAI 2015: 18th International Conference, Munich, Germany, October 5-9, 2015, Proceedings, Part III 18*. Springer, 234–241.

[33] Rishov Sarkar et al. 2023. Edge-moe: memory-efficient multi-task vision transformer architecture with task-level sparsity via mixture-of-experts. *arXiv preprint arXiv:2305.18691*.

[34] Rishov Sarkar et al. 2023. Edge-moe: memory-efficient multi-task vision transformer architecture with task-level sparsity via mixture-of-experts. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE.

[35] Yifan Song et al. 2022. SAUST: A Scheme for Acceleration of Unstructured Sparse Transformer. In *2022 IEEE International Conference on Integrated Circuits, Technologies and Applications (ICTA)*. (Oct. 2022), 56–57.

[36] Robin Strudel et al. 2021. Segmenter: transformer for semantic segmentation. In *Proceedings of the IEEE/CVF international conference on computer vision*.

[37] Mengshu Sun et al. 2022. Vaqf: fully automatic software-hardware co-design framework for low-bit vision transformer. *arXiv preprint arXiv:2201.06618*.

[38] Hugo Touvron et al. 2021. Training data-efficient image transformers & distillation through attention. In *Proceedings of the 38th International Conference on Machine Learning*. PMLR, (July 1, 2021), 10347–10357. Retrieved Jan. 14, 2024 from https://proceedings.mlr.press/v139/touvron21a.html.

[39] Fengbin Tu et al. 2022. Trancim: full-digital bitline-transpose cim-based sparse transformer accelerator with pipeline/parallel reconfigurable modes. *IEEE Journal of Solid-State Circuits*.

[40] Shikhar Tuli et al. 2023. Acceltran: a sparsity-aware accelerator for dynamic inference with transformers. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*.

[41] Erwei Wang et al. 2020. LUTNet: Learning FPGA Configurations for Highly Efficient Neural Network Inference. *IEEE Transactions on Computers*, 69, 12, (Dec. 2020), 1795–1808.

[42] Teng Wang et al. 2022. Via: a novel vision-transformer accelerator based on fpga. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 41, 11, 4088–4099.

[43] Shaarada D. Yamini et al. 2023. Hardware Accelerator for Transformer based End-to-End Automatic Speech Recognition System. In *2023 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. (May 2023).

[44] Xin Yang et al. 2022. Efa-trans: an efficient and flexible acceleration architecture for transformers. *Electronics*, 11, 21, 3550.

[45] Haoran You et al. 2023. ViTCoD: Vision Transformer Acceleration via Dedicated Algorithm and Accelerator Co-Design. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. (Feb. 2023), 273–286.

[46] Xiaofan Zhang et al. 2018. Dnnbuilder: an automated tool for building high-performance dnn hardware accelerators for fpgas. In *2018 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. ACM, 1–8.

[47] Yuan Zhang et al. 2022. Base-2 softmax function: suitability for training and efficient hardware implementation. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 69, 9, 3605–3618.

[48] Yiren Zhao et al. 2019. Automatic Generation of Multi-Precision Multi-Arithmetic CNN Accelerators for FPGAs. In *2019 International Conference on Field-Programmable Technology (ICFPT)*. (Dec. 2019), 45–53.

[49] Jinming Zhuang et al. 2024. Ssr: spatial sequential hybrid architecture for latency throughput tradeoff in transformer acceleration. In *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 55–66.